

软件测试课程实验报告——实验二

专 业：软将工程
班 级：14012303
姓 名：梁景毓
学 号：2023302855

1.1 实验要求

本次实验旨在通过黑盒测试方法，对电子飞镖靶计分系统的核心功能和通信接口进行全面验证。

- 功能验证：确保系统能够正确处理飞镖类型和数量的输入，并根据靶区命中信息准确计分。
- 边界值分析：测试系统对输入数据边界条件（如最大飞镖数量）的处理能力。
- 接口容错测试：检验通信协议的稳定性和系统的异常处理能力，确保在接收到格式错误、内容无效的数据包时，系统不会崩溃或产生意外行为。
- 实验环境：
 - 硬件：电子飞镖靶模拟系统
 - 软件：ETest 嵌入式测试平台
 - 脚本语言：Lua

1.2 实验过程

1.2.1 飞镖信息输入功能测试 (1shoot.lua)

本次测试使用了 `project/testdata` 目录下的 `data_biao.yml` `data_biao2.yml`，和 `data_biao3.yml` 进行全覆盖数据驱动测试。

测试用例与结果分析

测试集	需求场景	关键测试数据	预期输出	实际输出	问题描述
<code>data_biao.yml</code>	轻镖 (Type 1) 正常值 范围 (1-20)	<code>{type: 1, number: 10}</code>	倒计时 = 数量 × 3秒 (10枚对应30秒)	倒计时准确更新为30秒, 步进正确	验证通过。 系统倒计时逻辑与需求规格说明书一致。
<code>data_biao.yml</code>	轻镖 (Type 1) 边界值 (1, 20)	<code>{type: 1, number: 1}</code> , <code>{type: 1, number: 20}</code>	1枚显示3秒; 20枚显示60秒	边界值处理正确, 分别显示3秒和60秒	验证通过。 边界条件处理平滑, 未见溢出或计算错误。
<code>data_biao.yml</code>	轻镖 (Type 1) 越界值 (>20)	<code>{type: 1, number: 21}</code> , <code>{number: 34}</code>	系统按最大值20处理, 倒计时60秒	输入21、34等越界值时, 系统均锁定为60秒, 截断逻辑生效	验证通过。 输入截断保护机制有效, 防止了非法长时倒计时。
<code>data_biao.yml</code>	轻镖 (Type 1) 零值 (0)	<code>{type: 1, number: 0}</code>	系统不响应或提示错误	系统无任何倒计时更新, 维持原状态	验证通过。 无效输入 (0枚) 被系统前端正确忽略。
<code>data_biao2.yml</code>	重镖 (Type 2) 正常值 范围 (1-20)	<code>{type: 2, number: 20}</code>	倒计时 = 数量 × 4.5秒 (20枚对应90秒)	倒计时准确更新为90秒	验证通过。 重镖倍率 (4.5s/枚) 计算准确。
<code>data_biao2.yml</code>	重镖 (Type 2) 越界值 (>20)	<code>{type: 2, number: 21}</code> , <code>{number: 34}</code>	系统按最大值20处理, 倒计时90秒	输入越界值时, 系统正确截断, 倒计时显示90秒	验证通过。 重镖模式下的上限截断逻辑工作正常。
<code>data_biao3.yml</code>	无效飞镖类型 遍历	<code>{type: 0, 3, 4, ..., 255}</code>	系统丢弃无效类型包, 无响应	遍历所有定义外的Type值, 系统均保持静默, 无误触发	验证通过。 系统具备良好的抗干扰能力, 只响应合法类型。

对应测试代码

```
1  -- 飞镖信息输入功能测试
2  function Shoot()
3      -- 检查是否存在外部导入的测试数据 (data.yml)
4      if test_data and test_data.shoot then
5          -- 模式1: 数据驱动测试
6          for _index, value in ipairs(test_data.shoot) do
7              local type_val = value.type
8              local num_val = value.number
9          end
10     end
11 end
```

```
10     print(string.format(">>> 执行YAML用例 %d: 飞镖类型=%d, 数量=%d",
11 _index, type_val, num_val))
12
13     -- 发送协议数据
14     write_msg(channels.shoot, protocols.shoot, {type = type_val,
15 number = num_val})
16
17     -- 延时 1000ms 以便观察软件反应 (如倒计时是否更新)
18     etimer.delay(1000)
19 end
20 else
21     -- 模式2: 默认全覆盖测试 (当没有读取到数据时)
22     print("警告: 未读取到 data.yml 数据, 执行内置默认逻辑")
23
24     -- 场景1: 轻镖 (Type=1), 10枚 -> 预期时间 30秒
25     print(">>> 默认场景1: 轻镖 (Type 1), 10枚 (预期: 30秒)")
26     write_msg(channels.shoot, protocols.shoot, {type = 1, number = 10})
27     etimer.delay(2000)
28
29     -- 场景2: 重镖 (Type=2), 20枚 -> 预期时间 90秒
30     print(">>> 默认场景2: 重镖 (Type 2), 20枚 (预期: 90秒)")
31     write_msg(channels.shoot, protocols.shoot, {type = 2, number = 20})
32     etimer.delay(2000)
33
34     -- 场景3: 数量边界测试 (21枚) -> 预期截断为 20 [需求文档 2.2]
35     print(">>> 默认场景3: 边界测试 (21枚) (预期: 截断为20)")
36     write_msg(channels.shoot, protocols.shoot, {type = 1, number = 21})
37     etimer.delay(1000)
38 end
39 end
```

1.2.2 靶区命中信息接收功能测试 (2target.lua)

本次测试使用了 `project/testdata/data.yml` 进行测试，并结合脚本循环覆盖了所有有效靶区。

测试用例与结果分析

需求编号	测试场景	输入数据	预期输出	实际输出	问题描述
Fun-Target-01	全靶区遍历测试 (Zone 1-5)	<code>write_msg</code> 循环发送 <code>{zone = 1..5}</code> ，每区3次	系统依次识别：内牛眼、外牛眼、三倍区、双倍区、单倍区，并对应累加分数	各靶区命中后，界面分数实时刷新，分值计算逻辑正确	验证通过。五种靶区类型的分值定义和映射关系无误。
Fun-Target-02	(数据驱动) 单一靶区连续命中	<code>data.yml</code> ： <code>[[{zone:1}, {zone:1}, {zone:1}]]</code>	连续3次命中内牛眼，分数应累加 3 × Zone1分值	连续发送3帧数据，系统响应敏捷，总分准确增加，无丢包现象	验证通过。高频连发场景下系统处理能力稳定，积分逻辑正确。

对应测试代码

```
1 -- 接收靶区信息功能测试
2 function Target()
3     -- 遍历靶区 1 到 5 (需求文档定义: 1-内牛眼, 2-外牛眼, 3-三倍, 4-双倍, 5-单倍)
4     for zone_id = 1, 5 do
```

```
5 print(string.format("=== 开始测试靶区: %d ===", zone_id))
6
7 -- 每个靶区连续发送 3 次，观察分数累加
8 for i = 1, 3 do
9     write_msg(channels.target, protocols.target, {zone = zone_id})
10    -- 发送间隔 500ms，模拟真实投镖间隔
11    etimer.delay(500)
12 end
13
14 -- 切换下一个区域前等待，方便观察界面变化
15 etimer.delay(1000)
16 end
17
18 print("=== 靶区循环测试结束 ===")
19 end
```

1.2.3 飞镖信息输入接口容错测试 (3interface_shoot.lua)

测试用例描述

需求编号	测试用例描述	输入数据	预期输出	实际输出	问题描述
IF-Shoot-00	正向测试 (Happy Path)	<code>write_msg</code> 发送 <code>{type=1, number=10}</code>	系统正常接收并处理，界面更新飞镖信息。	界面倒计时立即更新为30秒，通信指示灯闪烁。	无异常。功能响应正常。
IF-Shoot-01	包头错误	<code>write_msg</code> 发送 <code>{header=0x5511}</code>	系统丢弃该数据包，无任何响应。	界面无变化，倒计时未更新，系统保持静默。	校验通过。错误包头未触发系统误动作。
IF-Shoot-02	数据标志错误	<code>write_msg</code> 发送 <code>{data=0xAA}</code>	系统丢弃该数据包，无任何响应。	界面维持原状，未见异常跳转或显示乱码。	校验通过。无效标志位被底层协议栈正确过滤。
IF-Shoot-03	包尾错误	<code>write_msg</code> 发送 <code>{tail=0x5511}</code>	系统丢弃该数据包，无任何响应。	系统无响应，通信状态正常，未进入死锁。	校验通过。帧结构错误处理机制有效。
IF-Shoot-04	校验和错误	<code>write_msg</code> 发送 <code>{check=0}</code>	系统丢弃该数据包，无任何响应。	接收端忽略该数据帧，界面参数未发生改变。	校验通过。校验和验证逻辑工作正常，防止了脏数据录入。

对应测试代码

```
1 -- 数据传送机接口容错测试
```

```

2 function Shoot()
3     -- Test 0: 正向测试 (Happy Path)
4     print("测试项0: 发送正确数据包 (验证链路正常)")
5     write_msg(channels.shoot, protocols.shoot, {type=1, number=10})
6     etimer.delay(1000)
7     local res = ask("yesno", {title='提示', msg='请确认是否收到了飞镖信息
      (类型/数量更新)? \n(预期: 是)', default=true})
8     check(res, "发送正确数据包成功", "发送正确数据包失败")
9
10    -- Test 1: 包头错误
11    print("测试项1: 发送错误包头 (0x5511)")
12    write_msg(channels.shoot, protocols.shoot, {header=0x5511})
13    etimer.delay(1000)
14    local res = ask("yesno",{title='提示', msg='请确认是否收到了飞镖信息输入
      \n(预期: 否 - 应丢包)', default=false})
15    check(not res, "包头错误有丢包处理, 正确", "包头错误没有丢包处理, 错误")
16
17    -- Test 2: 数据标志错误
18    print("测试项2: 发送错误数据标志 (0xAA)")
19    write_msg(channels.shoot, protocols.shoot, {data=0xAA})
20    etimer.delay(1000)
21    local res = ask("yesno",{title='提示', msg='请确认是否收到了飞镖信息输入
      \n(预期: 否 - 应丢包)', default=false})
22    check(not res, "数据标志错误有丢包处理, 正确", "数据标志错误没有丢包处理, 错误")
23
24    -- Test 3: 包尾错误
25    print("测试项3: 发送错误包尾 (0x5511)")
26    write_msg(channels.shoot, protocols.shoot, {tail=0x5511})
27    etimer.delay(1000)
28    local res = ask("yesno",{title='提示', msg='请确认是否收到了飞镖信息输入
      \n(预期: 否 - 应丢包)', default=false})
29    check(not res, "包尾错误有丢包处理, 正确", "包尾错误没有丢包处理, 错误")
30
31    -- Test 4: 校验和错误
32    print("测试项4: 发送错误校验和 (0)")
33    write_msg(channels.shoot, protocols.shoot, {check=0})
34    etimer.delay(1000)
35    local res = ask("yesno",{title='提示', msg='请确认是否收到了飞镖信息输入
      \n(预期: 否 - 应丢包)', default=false})
36    check(not res, "校验和错误有丢包处理, 正确", "校验和错误没有丢包处理, 错误")
37 end

```

1.2.4 靶区命中信息接口容错测试 (4interface_target.lua)

测试用例描述

需求编号	测试用例描述	输入数据	预期输出	实际输出	问题描述
IF-Targ-00	正向测试 (Happy Path)	<code>write_msg</code> 发送 <code>{zone=1}</code>	系统正常接收并处理，界面更新分数。	界面立即显示“内牛眼”得分动画，总分增加。	无异常。正向链路验证通过。
IF-Targ-01	校验和错误	<code>write_msg</code> 发送 <code>{check=0}</code>	系统丢弃该数据包，无任何响应。	界面分数无变化，未触发计分逻辑。	校验通过。错误校验和帧被正确丢弃。
IF-Targ-02	包头错误	<code>write_msg</code> 发送 <code>{header=0x5511}</code>	系统丢弃该数据包，无任何响应。	系统静默，界面无闪烁或误判。	校验通过。非法包头未被系统接纳。
IF-Targ-03	数据标志1错误	<code>write_msg</code> 发送 <code>{data1=0x55}</code>	系统丢弃该数据包，无任何响应。	界面维持原状，未见异常计分。	校验通过。标志位检查机制生效。
IF-Targ-04	无效靶区号 (逻辑错误)	<code>write_msg</code> 发送 <code>{zone=0x9}</code>	系统丢弃该数据包或无视该消息。	发送无效区域ID后，系统无动作，未崩溃。	校验通过。业务逻辑层成功拦截非法参数。
IF-Targ-05	数据标志2错误	<code>write_msg</code> 发送 <code>{data2=0x55}</code>	系统丢弃该数据包，无任何响应。	界面无响应，系统运行稳定。	校验通过。多重标志位验证机制有效。
IF-Targ-06	包尾错误	<code>write_msg</code> 发送 <code>{tail=0x5511}</code>	系统丢弃该数据包，无任何响应。	系统无响应，通信指示灯未异常闪烁。	校验通过。帧尾校验防止了残缺帧处理。

对应测试代码

```
1  -- 靶体接口容错测试
2  function Target()
3      -- Test 0: 正向测试 (Happy Path)
4      print("测试项0: 发送正确靶体数据 (Zone 1)")
5      write_msg(channels.target, protocols.target, {zone=1})
6      etimer.delay(1000)
7      local res = ask("yesno", {title='提示', msg='请确认界面是否显示击中"内牛眼"或分数增加? \n(预期: 是)', default=true})
8      check(res, "发送正确数据包成功", "发送正确数据包失败")
9
10     -- Test 1: 校验和错误
11     print("测试项1: 发送错误校验和 (0)")
12     write_msg(channels.target, protocols.target, {check=0})
13     etimer.delay(1000)
14     local res = ask("yesno",{title='提示', msg='请确认是否收到了靶体信息\n(预期: 否 - 应丢包)', default=false})
15     check(not res, "校验和错误有丢包处理, 正确", "校验和错误没有丢包处理, 错误")
16
17     -- Test 2: 包头错误
18     print("测试项2: 发送错误包头 (0x5511)")
```

```

19 write_msg(channels.target, protocols.target, {header=0x5511})
20 etimer.delay(1000)
21 local res = ask("yesno",{title='提示', msg='请确认是否收到了靶体信息\n(预期: 否 - 应丢包)', default=false})
22 check(not res, "包头错误有丢包处理, 正确", "包头错误没有丢包处理, 错误")
23
24 -- Test 3: 数据标志1错误
25 print("测试项3: 发送错误数据标志1 (0x55)")
26 write_msg(channels.target, protocols.target, {data1=0x55})
27 etimer.delay(1000)
28 local res = ask("yesno",{title='提示', msg='请确认是否收到了靶体信息\n(预期: 否 - 应丢包)', default=false})
29 check(not res, "数据标志1错误有丢包处理, 正确", "数据标志1错误没有丢包处理, 错误")
30
31 -- Test 4: 无效靶区号 (逻辑错误)
32 print("测试项4: 发送无效靶区号 (9)")
33 write_msg(channels.target, protocols.target, {zone=0x9})
34 etimer.delay(1000)
35 local res = ask("yesno",{title='提示', msg='请确认是否收到了靶体信息\n(预期: 否 - 应丢包)', default=false})
36 check(not res, "无效靶区号有丢包处理, 正确", "无效靶区号没有丢包处理, 错误")
37
38 -- Test 5: 数据标志2错误
39 print("测试项5: 发送错误数据标志2 (0x55)")
40 write_msg(channels.target, protocols.target, {data2=0x55})
41 etimer.delay(1000)
42 local res = ask("yesno",{title='提示', msg='请确认是否收到了靶体信息\n(预期: 否 - 应丢包)', default=false})
43 check(not res, "数据标志2错误有丢包处理, 正确", "数据标志2错误没有丢包处理, 错误")
44
45 -- Test 6: 包尾错误
46 print("测试项6: 发送错误包尾 (0x5511)")
47 write_msg(channels.target, protocols.target, {tail=0x5511})
48 etimer.delay(1000)
49 local res = ask("yesno",{title='提示', msg='请确认是否收到了靶体信息\n(预期: 否 - 应丢包)', default=false})
50 check(not res, "包尾错误有丢包处理, 正确", "包尾错误没有丢包处理, 错误")
51 end

```

1.3 思考总结

- **系统化测试思维**: 通过将测试分为功能测试和接口容错测试, 可以更结构化、更全面地覆盖系统功能点。功能测试保证“系统能做什么”, 接口测试保证“系统在异常时不出错”。
- **测试自动化**: 编写测试脚本 (如 `1shoot.lua`, `2target.lua`) 能够极大提高测试效率, 使得回归测试变得简单快捷。特别是接口容错测试, 手动构造大量错误报文非常繁琐, 自动化脚本优势明显。
- **测试技术应用**:
 - **边界值分析** (`1shoot.lua`): 对于有明确输入范围的 (如飞镖数量), 测试其边界内、外的值是发现错误的高效手段。
 - **错误注入** (`3interface_shoot.lua`, `4interface_target.lua`): 主动向系统发送格式错误、逻辑错误的数据, 是检验系统健壮性的关键。一个好的系统不仅要处理正确输入, 更要能优雅地处理异常输入。